ThreatWatch CyberLab

Software Design Document (SDD)

Gynia Prawl

Xavier University of Louisiana

CPSC 4800 - Capstone I

Dr. Edwards

April 25th, 2026

Table of Contents

1. Problem Statement

Many cybersecurity students and entry-level analysts have a hard time gaining real-world experience since they don't have access to realistic security environments. Enterprise security systems like Security Information and Event Management (SIEM) platforms and real Security Operations Center (SOC) infrastructures are typically hard to use, cost a lot of money, and are not easy for beginners to get to. Because of this, students may know a lot about cybersecurity in theory but not have any real-world experience with things like assessing alerts, responding to situations, and figuring out how defensive decisions affect things.

ThreatWatch CyberLab fills this gap by offering a lightweight, simulation-based environment for intrusion detection that lets users practice finding and dealing with cyber threats in a safe setting. The tool lets students practice their investigative and defensive abilities in a safe way by mimicking real-life attack situations.

2. User Personas

ThreatWatch CyberLab is designed for users with varying levels of cybersecurity knowledge who want to develop practical experience identifying and responding to cyber threats. The following personas represent typical users who would benefit from the system.

2.1 Persona 1 - Sekani Gray (Cybersecurity Intern)

Sekani Gray is a 23-year-old cybersecurity intern who recently earned a bachelor's degree in Cybersecurity. He has a strong foundation in security concepts such as networking, intrusion detection, and incident response, but limited real-world experience working inside a Security Operations Center (SOC).

As an intern, Sekani wants to build confidence by applying what he learned in school to realistic scenarios. He is motivated to improve his technical skills, better understand security alerts, and prepare for a full-time cybersecurity role.

Demographics

- **Age:** 23
- **Education:** Bachelor's degree in Cybersecurity
- **Job Role:** Cybersecurity Intern
- **Technical Skill Level:** Entry-level to intermediate

Goals

- Practice working with intrusion detection systems
- Learn how to interpret and prioritize security alerts
- Gain hands-on experience responding to cyber threats
- Build confidence before transitioning into a full-time role

Frustrations / Pain Points

- Most training is theoretical and not hands-on
- Real SOC tools can be overwhelming for beginners
- Limited opportunities to safely practice making mistakes
- Difficulty connecting classroom knowledge to real incidents

How Sekani Uses ThreatWatch CyberLab

Sekani utilizes ThreatWatch CyberLab as a hands-on training environment, where he can observe simulated cyberattacks and observe how an intrusion detection system responds. He reviews alerts, decides how to respond, and learns from feedback on his actions. This allows Sekani to practice real-world cybersecurity workflows without the risk of impacting a live system.

Why This Persona Matters

This persona represents an early-career cybersecurity professional who needs practical experience to bridge the gap between education and real-world work. Designing ThreatWatch CyberLab with Sekani in mind ensures the platform focuses on usability, learning, and realistic security workflows rather than unnecessary complexity.

2.2 Persona 2 - Christiana Love (Undergrad Student)

Christiana Love is a 20-year-old undergraduate Computer Science student who is interested in cybersecurity but has limited exposure to security concepts and tools. She has completed introductory programming and networking coursework but has not yet worked with intrusion detection systems, security logs, or incident response processes.

Christiana is curious about cybersecurity as a potential career path, but feels overwhelmed by the complexity of professional security tools. She wants a supportive, low-risk environment where she can learn how cyberattacks are detected and handled without needing advanced prior knowledge.

Demographics

- **Age:** 20
- **Education:** Undergraduate Computer Science major
- **Job Role:** Student (pre-internship)
- **Technical Skill Level:** Beginner

Goals

- Understand how cybersecurity works in real-world settings
- Learn basic intrusion detection and response concepts
- Gain hands-on experience without fear of breaking systems
- Build confidence before taking advanced security courses

Frustrations / Pain Points

- Security terminology and tools feel intimidating
- Learning resources often assume prior security knowledge
- Difficulty visualizing how attacks and defenses interact
- Fear of making mistakes while learning

How Christiana Uses ThreatWatch CyberLab

Christiana uses ThreatWatch CyberLab as an introductory training platform. She begins with guided scenarios that explain alerts in simple language and offer clear response options. Through

repeated exposure and feedback, Christiana learns to recognize common attack patterns and understand why certain responses are effective, helping her gradually build foundational cybersecurity skills.

Why This Persona Matters

Christiana represents beginner users who are exploring cybersecurity for the first time. Designing ThreatWatch CyberLab with Christiana in mind ensures the platform remains accessible, educational, and supportive, making cybersecurity concepts easier to understand for users without a strong technical background.

2.3 Persona 3 - Dr. Angel Ryder (Cybersecurity Instructor)

Dr. Angel Ryder is a cybersecurity instructor and professor who teaches undergraduate and early graduate-level security courses. She has a strong background in cybersecurity concepts and professional experience, but is primarily focused on helping students translate theory into real-world practice.

Dr. Ryder wants a safe, structured platform that allows students to experience intrusion detection and response workflows without the cost, complexity, or risk of enterprise security systems.

Demographics

- **Age:** 47
- **Education:** Master's in Cybersecurity
- **Job Role:** Cybersecurity Instructor / Trainer
- **Technical Skill Level:** Advanced

Goals

- Provide students with hands-on cybersecurity experience
- Demonstrate how intrusion detection systems work in practice
- Teach decision-making and response workflows, not just theory
- Evaluate student understanding through realistic scenarios

Frustrations / Pain Points

- Limited access to realistic SOC-style environments for teaching
- Enterprise tools are too complex or expensive for classroom use
- Traditional labs focus on exploitation rather than defense
- Difficulty assessing how well students understand response processes

How Dr. Angel Ryder Uses ThreatWatch CyberLab

Dr. Ryder uses ThreatWatch CyberLab as an instructional and training tool. She assigns scenarios to students, allows them to interact with simulated intrusions, and reviews performance summaries to assess how students interpret alerts and respond to incidents. The platform supports guided learning while allowing Dr. Ryder to scale hands-on practice across multiple students.

Why This Persona Matters

Dr. Ryder represents instructors and trainers who shape how cybersecurity is taught. Designing ThreatWatch CyberLab with her needs in mind ensures the platform supports structured learning, guided progression, and assessment, making it suitable for classroom and training environments.

3. Functional Requirements

The following functional requirements outline the primary functionalities and behaviors of the ThreatWatch CyberLab system. These requirements explain how users will use the platform and how the system will operate in simulated cybersecurity situations.

FR-01: Intrusion Type Support

The system shall simulate and detect up to three categories of intrusion events: network-based intrusion, host-based intrusion, and password-based intrusion.

Rules / Details

- Each intrusion event shall be assigned one category.
- The category shall be visible to the user during investigation.

Acceptance Criteria

- Given a session is active, when an intrusion is generated, then it is classified as one of the supported types.
- Given an alert is opened, then the intrusion type is displayed.

FR-02: Attack Notification

When a simulated intrusion event is triggered, the system shall generate and display an alert notification to the user.

Rules / Details

- Each alert shall include:
    - intrusion type

- severity level
- timestamp
- short description

- Alerts shall appear in the Active Alerts panel.

Acceptance Criteria

- Given an intrusion occurs, when it is detected, then an alert appears in the Active Alerts panel.
- Given an alert appears, then it includes type, severity, and timestamp.

FR-03: Training Profile Selection

Before starting a session, the system shall require the user to select a training profile and configure the session accordingly.

Rules / Details

- Available profiles shall include:
  - Sekani Gray (intermediate)
  - Christiana Love (beginner)
  - Dr. Angel Ryder (instructor)
- The system shall adjust guidance level based on the selected profile.
- The session shall not begin until a profile is selected.

Acceptance Criteria

- Given no profile is selected, when the user attempts to start a session, then the system prevents session start.

- Given a profile is selected, when the session begins, then the system applies the corresponding guidance level.

FR-04: Randomized Attack Triggering

During an active session, the system shall trigger intrusion events at random intervals.

Rules / Details

- Events shall not require manual input to occur.
- Timing between events shall vary.

Acceptance Criteria

- Given a session is active, when time progresses, then intrusion events occur without user action.

FR-05: Investigation Capability

When an alert is displayed, the system shall allow the user to investigate the intrusion event and view its details.

Rules / Details

- Investigation view shall display:
  - intrusion type
  - severity level
  - indicators
  - current status

Acceptance Criteria

- Given an alert exists, when the user selects Investigate, then the system displays detailed intrusion information.

FR-06: Response Capability

After investigating an intrusion, the system shall allow the user to select a response to the event.

Rules / Details

- Only one response may be selected at a time.
- A response must be selected before resolution.

Acceptance Criteria

- Given an incident is active, when the user selects a response, then the system processes the selection.

FR-07: Alert Dismissal

The system shall allow the user to dismiss an alert without resolving the intrusion.

Rules / Details

- Dismissed alerts shall be removed from the Active Alerts panel.
- Dismissal may impact performance tracking.

Acceptance Criteria

- Given an alert exists, when the user selects Dismiss, then the alert is removed from the active list.

FR-08: Multiple Response Options

For each intrusion event, the system shall present multiple response options.

Rules / Details

- At least three response options shall be displayed.
- Options shall vary depending on intrusion type.

Acceptance Criteria

- Given an intrusion event, when the response screen is shown, then multiple response options are available.

FR-09: Attack Escalation

If the user selects an incorrect response or fails to respond within a given time, the system shall escalate the intrusion event.

Rules / Details

- Escalation shall:
  - increase severity level
  - update intrusion description
- Additional alerts may be generated.

Acceptance Criteria

- Given an incorrect response is selected, when evaluation completes, then severity increases.
- Given no response is selected, when time expires, then escalation occurs.

FR-10: Outcome Display

After a response is evaluated, the system shall display the outcome of the user's decision.

Rules / Details

- Outcome shall indicate:
  - contained
  - partially contained
  - escalated

Acceptance Criteria

- Given a response is selected, when evaluation completes, then the system displays the result.

FR-11: Explanation Feedback

After displaying the outcome, the system shall provide an explanation describing why the selected response succeeded or failed.

Rules / Details

- Explanation shall be simplified for beginner profiles.
- Explanation shall be concise for advanced users.

Acceptance Criteria

- Given an outcome is shown, when the explanation appears, then it describes the reasoning behind the result.

FR-12: Guided Learning Support

The system shall provide guided instructional support during training sessions based on the selected profile.

Rules / Details

- Beginner profile shall include:
  - hints
  - simplified explanations
- Advanced profile shall include minimal guidance.

Acceptance Criteria

- Given a beginner profile is selected, when interacting with alerts, then guidance is visible.
- Given an advanced profile is selected, then guidance is reduced.

FR-13: Points Reward System

When the user selects a correct response, the system shall award points.

Rules / Details

- Points shall be awarded once per resolved incident.
- Partially correct responses may award fewer points.

Acceptance Criteria

- Given a correct response is selected, when evaluation completes, then points are added to the user's total.

FR-14: Progress Tracking

During a session, the system shall track and update user performance metrics.

Rules / Details

- Metrics shall include:
  - total incidents
  - correct responses
  - incorrect responses
  - total points

Acceptance Criteria

- Given an incident is completed, when results are processed, then progress metrics update.

FR-15: Streak Tracking

The system shall track consecutive correct responses and update a streak count.

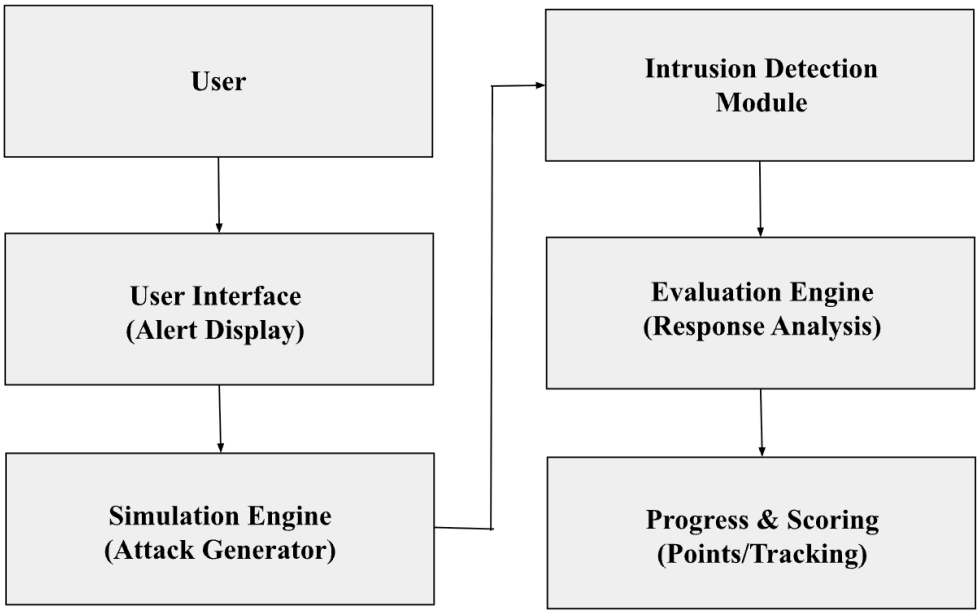
Rules / Details

- Streak increases with each correct response.
- Streak resets after an incorrect response.

Acceptance Criteria

- Given consecutive correct responses, when results are processed, then streak count increases.
- Given an incorrect response, when evaluated, then streak resets.

4. System Architecture



ThreatWatch CyberLab was constructed in a modular way, with many parts that work together. The user interface shows notifications and lets users look into and respond to fake attacks. This is how the user interacts with the system. The simulation engine makes up cyberattack scenarios that the intrusion detection module then works on. The system sends notifications to the user when it sees a threat. The user then has to look at the situation and decide how to respond. The evaluation engine looks at the user's answer and decides if the action worked. Finally, the progress and scoring module keeps track of how well users are doing, gives points for right answers, and keeps track of how far they've come in their training.

4.1 Network Architecture

ThreatWatch CyberLab operates as a host-based cybersecurity training system deployed within a single local machine environment. The system is designed to simulate both host-based and network-based intrusion scenarios without requiring a live external network. This approach ensures a controlled, safe, and lightweight environment while still maintaining realistic representations of cybersecurity events.

System Environment

All core components of ThreatWatch CyberLab execute within the same host system. These components include:

- Simulation Engine
- Intrusion Detection Module (IDM)
- Evaluation Engine
- User Interface
- In-memory data structures for session management

The system is implemented in Java and runs within a local development environment. It interacts directly with the host operating system through terminal-based command execution.

Architectural Context

As illustrated in Section 4, ThreatWatch CyberLab follows a modular architecture where components communicate through a structured internal data flow. Unlike distributed systems or enterprise security platforms, this communication does not occur over a physical network. Instead, all interactions take place within the application and host system boundary.

External System Interaction

Although the system operates locally, it integrates with operating system-level tools to simulate real-world intrusion detection scenarios. These tools act as external data sources for the Intrusion Detection Module.

Examples include:

- **AIDE (Advanced Intrusion Detection Environment):** Used for detecting file integrity changes
- **System logs:** Used to identify authentication failures and suspicious activity
- **Network utilities (e.g., netstat):** Used to monitor simulated connection activity

The system executes these tools via command-line instructions and captures their output. This output is then parsed and transformed into structured intrusion events for further processing within the application.

Network Simulation

ThreatWatch CyberLab simulates network-based activity to enhance realism in training scenarios. This includes:

- Source IP addresses
- Repeated connection attempts
- Network traffic patterns

These values may appear in system inputs and alerts (e.g., failed login attempts from a specific IP address), but they are generated or derived locally. They are not associated with actual

external network traffic. Instead, they are used to mimic real-world attack behavior within a controlled environment.

System Boundary

The system boundary is defined as the local host machine. All processing, detection, evaluation, and user interaction occur within this boundary.

External tools such as AIDE and system logs operate at the operating system level but remain part of the same environment. The system does not depend on external servers, cloud infrastructure, or distributed network components.

Communication Model

Communication between system components occurs through direct method calls and structured data objects within the Java application. Each module passes its output as input to the next component in the processing pipeline.

No external APIs, network protocols, or inter-process communication mechanisms are required for internal system operation. This design supports real-time processing, consistency, and simplicity.

4.2 Intrusion Detection Module Design

The Intrusion Detection Module is responsible for analyzing simulated attack events generated by the Simulation Engine and determining whether the activity represents a cybersecurity threat. This module acts as the detection layer of the ThreatWatch CyberLab platform.

Input

The module receives simulated attack event data from the Simulation Engine. Each event contains structured information describing the simulated network activity. The input may include the following attributes:

- Attack type
- Source IP address
- Target system or port
- Number of connection attempts
- Timestamp of the activity
- Event identifier

Processing Steps

When an attack event is received, the Intrusion Detection Module performs the following steps:

1. The module receives the simulated attack event from the Simulation Engine.
2. The module parses the event data and extracts key attributes such as attack type, source IP address, and connection attempts.
3. The module compares the event characteristics against predefined detection rules.
4. If the event matches a known intrusion pattern, the module classifies the event as a detected intrusion.
5. The module assigns a severity level (Low, Medium, High) based on the characteristics of the attack.

Output

After detection and classification, the module generates an intrusion alert that is sent to the Evaluation Engine. The output includes:

- Intrusion type
- Severity level
- Source IP address
- Event identifier
- Detection timestamp

The Evaluation Engine then uses this information to present the alert to the user and evaluate the user's response to the simulated attack.

4.3 Alert Generation Mechanism

Alert generation in ThreatWatch CyberLab is a deterministic process that originates directly from the output of the Intrusion Detection Module (IDM).

When the Simulation Engine generates an intrusion event, the event is passed to the IDM for analysis. The IDM evaluates the event against predefined detection rules and determines whether the activity represents a valid intrusion.

If the event is classified as a threat, the IDM produces a structured detection output. This output serves as the sole input for alert generation.

The system then constructs an alert object using the IDM output. Each alert contains the following attributes:

- intrusion type

- severity level
- timestamp
- event identifier
- descriptive summary of the detected activity

Alert generation does not occur independently and is not triggered by user actions. All alerts are exclusively derived from validated detection results produced by the IDM.

Once generated, the alert is immediately:

1. Stored within the system's active alert list
2. Displayed to the user through the User Interface
3. Passed to the Evaluation Engine for response tracking

This ensures that all alerts within the system are consistent, traceable, and directly linked to a specific intrusion detection event.

4.4 AIDE Integration (Detection Layer)

The ThreatWatch CyberLab system utilizes the open-source AIDE (Advanced Intrusion Detection Environment) configuration provided by Indiana University (2020) as the core detection layer. This component is responsible for identifying unauthorized changes to system files and generating data used to simulate intrusion events within the training environment.

AIDE Commands Used

The following commands are used to support intrusion detection:

- **Initialize database:** aide --init
- **Run integrity check:** aide --check
- **Update database after trusted changes:** aide --update

Input

The Intrusion Detection Module receives:

- Simulated system file changes generated by the Simulation Engine
- File attributes including:
  - File name
  - File size
  - Permissions
  - Checksum values

Processing Steps

1. The system executes `aide --check` to compare current file states with the baseline database
2. AIDE analyzes file attributes and identifies discrepancies
3. Detected changes are parsed and categorized as potential intrusion events
4. The system maps detected changes to intrusion types:
   - File modification → Host-based intrusion
   - Unauthorized access attempts → Password-based intrusion
5. Relevant detection data is sent to the Evaluation Engine

Output

The Intrusion Detection Module produces:

- Alerts indicating detected anomalies
- File-level change reports (e.g., checksum mismatches)
- Structured event data used to:
  - Trigger user alerts
  - Populate investigation details
  - Evaluate user responses

Mapping to Functional Requirements

Functional Requirement	AIDE Command	Description
FR-01: Intrusion Type Support	aide --init	Establishes baseline database required for detecting file-based intrusions
FR-01: Intrusion Type Support	aide --check	Detects file-based intrusions
FR-02: Attack Notification	aide --check	Generates data used for alerts
FR-05: Investigation Capability	aide --check	Provides file attribute details
FR-06: Response Capability	aide --check	Supplies data for decision-making
FR-09: Attack Escalation	aide --check	Identifies unresolved anomalies for escalation
FR-06: Response Capability	aide --update	Updates baseline after trusted changes based on user/system decisions

4.5Tools and Technologies

ThreatWatch CyberLab is implemented using a lightweight and accessible technology stack to support simulation-based cybersecurity training.

- **Programming Language:** Java (core application logic)
- **Development Environment:** IntelliJ IDEA
- **Intrusion Detection Tool:** AIDE (Advanced Intrusion Detection Environment)
- **Command-Line Utilities:** sha256sum, grep, netstat, system logs
- **Version Control:** Git/GitHub
- **Execution Environment:** Local machine (macOS/Linux terminal)
- **Data Storage:** Lightweight internal data structures (no external database required)

4.6 Data Management and System Data Structure

ThreatWatch CyberLab does not rely on a traditional external database management system. Instead, it uses structured, in-memory data representations during runtime to manage system state, user interactions, and simulated intrusion events. This approach supports the lightweight and self-contained nature of the application while still enabling accurate tracking of user performance and system behavior.

Data is generated dynamically by the Simulation Engine and processed through the Intrusion Detection Module (IDM), Evaluation Engine, and Scoring components. The system maintains temporary data structures during execution to support real-time decision-making, feedback, and progression.

Core Data Entities

User

Represents the active participant interacting with the system.

- user_id (unique identifier)
- profile_type (beginner, intermediate, instructor)
- score (numeric performance value)
- progress_level (current stage or difficulty)
- streak_count (consecutive correct responses)

Intrusion Event

Represents a simulated attack generated by the Simulation Engine.

- event_id
- intrusion_type (password, network, host-based)
- severity_level (low, medium, high)
- timestamp (time of occurrence)
- source (simulation-generated or AIDE output)

Alert

Represents a processed notification derived from an intrusion event.

- alert_id
- linked_event_id
- severity_level
- status (active, dismissed, resolved)
- description (brief explanation of detected activity)

Response

Represents the user's action in response to an alert.

- response_id
- alert_id
- selected_action (investigate, respond, dismiss)
- correctness (true/false)
- evaluation_result (correct, incorrect, partial)

Performance Data

Tracks overall user outcomes and system evaluation metrics.

- total_incidents
- correct_responses
- incorrect_responses
- total_score

Data Flow Overview

Data within the system follows a structured pipeline:

1. The Simulation Engine generates an intrusion event
2. The event is passed to the IDM for analysis
3. The IDM produces a detection result, which is converted into an Alert
4. The user interacts with the alert, producing a Response
5. The Evaluation Engine processes the response and updates Performance Data

Design Considerations

- All data is temporary and session-based, meaning it is not permanently stored unless extended in future versions.
- The system prioritizes real-time processing and feedback over long-term storage.
- The structure is intentionally lightweight to align with the system's goal as a training and simulation platform, rather than a production security monitoring system.

4.7 System Workflow

The ThreatWatch CyberLab system operates through a structured, sequential workflow that simulates intrusion scenarios, processes detection events, and evaluates user responses in real time. Each system component contributes to a continuous pipeline, ensuring that simulated attacks are detected, presented to the user, and resolved through guided interaction.

Workflow Sequence

Step 1: Intrusion Scenario Generation

The Simulation Engine initiates a cybersecurity event by generating a simulated intrusion scenario.

- Examples include:
  - password brute-force attempts
  - suspicious network activity
  - unauthorized file modifications

These events serve as the initial input to the system.

Step 2: Intrusion Detection Processing

The generated event is passed to the Intrusion Detection Module (IDM).

- The IDM:
  - analyzes the event or system output (e.g., AIDE results)
  - identifies whether the activity represents a threat
  - classifies the intrusion type
  - assigns a severity level

The IDM produces a structured detection result.

Step 3: Alert Generation

If a threat is detected, the system generates an alert based on the IDM's output.

- Each alert includes:
  - intrusion type
  - severity level
  - timestamp
  - description of the activity

The alert is then presented to the user for action.

Step 4: User Interaction

The user is prompted to respond to the alert.

Available actions include:

- **Investigate**
- **Respond**
- **Dismiss**

The selected action is captured as a structured response within the system.

Step 5: Response Evaluation

The user's response is processed by the Evaluation Engine.

- The system determines:
  - whether the response is correct

- whether it partially addresses the threat
- whether it fails to mitigate the intrusion

The evaluation result is recorded and used to update system state.

Step 6: Escalation Handling

If the response is incorrect or insufficient, the system applies escalation logic.

Escalation may include:

- increasing the severity level
- introducing additional attack indicators
- generating follow-up alerts
- simulating further system compromise

This step reinforces learning through consequence-based progression.

Step 7: Performance Tracking

The system updates user performance data based on the evaluation outcome.

- Metrics include:
  - score updates
  - correct vs. incorrect responses
  - Streak progression

This ensures continuous feedback and measurable progress throughout the session.

Workflow Summary

The system operates as a continuous pipeline:

Simulation Engine → Intrusion Detection Module → Alert Generation → User Interaction → Evaluation Engine → Escalation → Performance Tracking

Design Characteristics

- The workflow is sequential and dependency-driven, meaning each step relies on the output of the previous component.
- All processing occurs in real time, enabling immediate feedback and interaction.
- The system is designed as a closed-loop training environment, where each user action directly influences subsequent system behavior.

The evaluation and escalation processes described in Sections 4.7 and 4.8 govern how the system responds to user actions.

4.8 Evaluation Logic

The Evaluation Engine is responsible for determining the effectiveness of a user's response to a detected intrusion event. This process is rule-based and operates on structured data produced during alert generation and user interaction.

Input to Evaluation Engine

The Evaluation Engine receives:

- **Alert Data**
  - intrusion type
  - severity level
  - event identifier
- **User Response**
  - selected action (investigate, respond, dismiss)

- response timing (if applicable)

Evaluation Process

The Evaluation Engine compares the user's selected action against a predefined set of expected responses associated with each intrusion scenario.

Each scenario defines:

- correct actions
- partially correct actions
- incorrect actions

Evaluation Outcomes

The system classifies the user's response into one of three categories:

- **Correct**
  - the selected action fully mitigates the intrusion
  - the system marks the incident as resolved
- **Partially Correct**
  - the response addresses part of the threat
  - the system may continue the scenario with additional prompts
- **Incorrect**
  - the response fails to address the intrusion
  - escalation logic is triggered

System Actions Based on Evaluation

Following evaluation, the system:

- updates user score
- records the response outcome
- updates performance metrics
- determines whether escalation is required

Design Characteristics

- Evaluation is deterministic and scenario-driven, meaning outcomes are based on predefined rules rather than randomness
- All evaluation decisions are traceable to specific intrusion events and user actions
- The Evaluation Engine operates in real time, ensuring immediate feedback to the user

4.9 Escalation Logic

Escalation logic governs how the system responds when an intrusion is not properly handled by the user. It ensures that unresolved or incorrectly handled threats evolve in complexity, simulating real-world cybersecurity consequences.

Escalation Triggers

Escalation is triggered under the following conditions:

- the user selects an incorrect response
- the user selects a dismiss action for a valid threat
- the user fails to respond within a defined interaction window (if applicable)

Escalation Process

When triggered, the system modifies the current scenario using one or more of the following actions:

- **Increase severity level**
  - low → medium → high
- **Introduce additional indicators**
  - new suspicious activity
  - expanded system impact
- **Generate follow-up alerts**
  - linked to the original intrusion event
- **Simulate system compromise**
  - reflects the consequences of unresolved threats

Escalation Outcomes

Escalation results in:

- increased difficulty of the scenario
- additional decision-making steps for the user
- continued evaluation cycles until the intrusion is resolved

Integration with Evaluation Engine

Escalation logic is directly dependent on the output of the Evaluation Engine:

- incorrect → escalation triggered
- partial → possible escalation
- correct → no escalation

Design Characteristics

- Escalation is progressive, meaning system response intensifies over time
- Escalation is deterministic, based on user actions and predefined rules

- Each escalation event remains linked to the original intrusion event, ensuring traceability

4.10 System Integration

System integration in ThreatWatch CyberLab defines how all core components operate together as a unified system. Each module is designed to function independently but is connected through a structured data flow pipeline, where the output of one component becomes the input of the next.

The system follows a tightly coupled execution sequence:

- The Simulation Engine generates an intrusion event
- The event is passed to the Intrusion Detection Module (IDM)
- The IDM produces a structured detection output
- The detection output is used to generate an alert
- The alert is displayed to the user via the User Interface
- The user's selected response is captured by the system
- The Evaluation Engine processes the response
- The Escalation Logic determines whether additional actions are required
- The Progress & Scoring module updates system state

Each integration point is deterministic and explicitly defined, ensuring that no component operates in isolation. All transitions between components are based on structured data outputs, which guarantees consistency, traceability, and predictable system behavior.

Integration is implemented programmatically within the application using method calls and controlled execution flow, rather than external messaging systems or APIs. This aligns with the system's lightweight design and ensures efficient real-time processing.

The integration design ensures that:

- all system components are synchronized
- data flows continuously without interruption
- user actions directly influence system outcomes
- system behavior remains consistent across all scenarios

This integration model aligns directly with the system workflow (Section 4.6) and governs how evaluation and escalation decisions (Sections 4.7 and 4.8) are applied during execution.

5. Intrusion Detection Methods

5.1 Scenario #1 – File Integrity Change Detection

This scenario demonstrates a host-based intrusion detection approach using file integrity verification.

Objective: To demonstrate how the Intrusion Detection Module detects unauthorized changes to a file by comparing checksum values before and after modification.

Step 1: Input Creation (Initial File Setup)

A text file is created on the user's local machine to serve as the baseline input.

Steps:

1. Open terminal
2. Navigate to the desired directory
3. Create a new file:

touch sample.txt

- The touch command is used to create a new file named sample.txt.
- If the file does not exist, it is created as an empty file.
- If the file already exists, the command updates its timestamp.

4. Add initial content to the file:

echo "safe" > sample.txt

- The echo command writes text into the file.
- The > operator overwrites the file contents with the specified text.

Example Input (Initial File)

File Name: sample.txt

File Contents:

safe

Step 2: Generate Initial Checksum

The system computes a checksum of the file to establish its original state.

Command Used:

sha256sum sample.txt

- The sha256sum command generates a cryptographic hash (checksum) of the file.
- A checksum is a unique string that represents the exact contents of a file.

- SHA-256 is used because it provides strong integrity verification and ensures that even small changes in the file result in a completely different hash value.

Example Output (Initial Checksum)

5f70bf18a08660b7... sample.txt

(This value represents the baseline checksum of the file.)

Step 3: Modify the File (Simulated Intrusion)

The file is altered to simulate an unauthorized change.

Steps:

echo "hacked" > sample.txt

- This command overwrites the original contents of the file.
- The change represents a potential intrusion or unauthorized modification.

Example Input (Modified File)

File Name: sample.txt

File Contents:

hacked

Step 4: Generate New Checksum

The checksum is recalculated after the file has been modified.

Command Used:

sha256sum sample.txt

Example Output (Modified Checksum)

2cf24dba5fb0a30e... sample.txt

Step 5: Compare Checksums

The system compares the initial checksum with the modified checksum.

Observation:

- Initial checksum ≠ Modified checksum
- Because checksums uniquely represent file contents, any difference indicates that the file has been altered.

Step 6: Intrusion Detection

Because the checksum values differ, the system determines that the file has been modified.

Conclusion:

An intrusion event is detected due to unauthorized modification of the file.

Connection to Intrusion Detection Module

- The file acts as the system input
- The checksum represents file integrity
- A mismatch indicates a violation of integrity

This supports **FR-01: Intrusion Type Support (host-based intrusion detection)**

5.2 Scenario #2 – Password-Based Intrusion Detection

This scenario demonstrates how the Intrusion Detection Module detects unauthorized access attempts by monitoring failed authentication activity.

Objective: To demonstrate how the Intrusion Detection Module detects password-based intrusion attempts by identifying repeated failed login events.

Step 1: Input Creation (Unauthorized Access Attempt Setup)

An unauthorized access attempt is simulated by attempting to switch to another user account using incorrect credentials.

Steps:

1. Open terminal
2. Execute the following command:

```
su fakeuser
```

   - The su (switch user) command is used to attempt access to another user account.
   - The username fakeuser represents a non-existent or unauthorized account.
   - This ensures that authentication fails and the attempt is recorded in system logs.
3. Enter incorrect passwords multiple times when prompted
   - Repeated failed attempts simulate a password-based intrusion

Example Input (Unauthorized Attempt)

Username: fakeuser

Password Attempts:

- password123

- admin123

- letmein

Step 2: Generate System Log Data

Failed login attempts are automatically recorded by the operating system in authentication log files.

- These logs serve as the input for intrusion detection

Step 3: Detection Command Execution

The Intrusion Detection Module retrieves failed login attempts from system logs.

Command Used:

```
grep "Failed password" /var/log/auth.log
```

- The grep command searches for specific text within a file
- “Failed password” identifies unsuccessful authentication attempts
- /var/log/auth.log stores authentication activity on most Linux systems

Note: Log file location may vary by system.

Example Output (Failed Login Attempts)

Apr 16 10:15:32 system sshd[1234]: Failed password for invalid user fakeuser from 192.168.1.10 port 22 ssh2

Apr 16 10:15:35 system sshd[1235]: Failed password for invalid user fakeuser from 192.168.1.10 port 22 ssh2

Apr 16 10:15:38 system sshd[1236]: Failed password for invalid user fakeuser from 192.168.1.10 port 22 ssh2

Step 4: Output Analysis

The system processes the log entries to identify patterns of failed authentication.

Observation:

- Multiple failed login attempts detected
- Same username and source IP address
- Occurring within a short time interval

Step 5: Intrusion Detection

Repeated failed authentication attempts indicate suspicious behavior.

Conclusion:

An intrusion event is detected due to repeated unauthorized login attempts consistent with a password-based attack.

Connection to Intrusion Detection Module

- Authentication logs act as system input
- Failed login entries indicate access violations
- Repeated failures trigger detection logic
- Supports:
  - **FR-01: Intrusion Type Support (password-based intrusion)**
  - **FR-09: Attack Escalation**

5.3 Scenario #3 – Network-Based Intrusion Detection

This scenario demonstrates how the Intrusion Detection Module detects suspicious network activity by monitoring incoming connection attempts.

Objective: To demonstrate how the Intrusion Detection Module detects network-based intrusion attempts by identifying unusual or repeated connection activity.

Step 1: Input Creation (Simulated Network Activity)

A network-based intrusion is simulated by generating repeated connection attempts to the system.

Steps:

1. Open terminal
2. Run the following command to simulate connection attempts:

```
ping -c 5 localhost
```

- The ping command sends network packets to a target system

- -c 5 sends 5 packets
- localhost represents the local machine

This simulates repeated network activity directed at the system

Example Input (Network Activity)

Target: localhost

Packets Sent: 5

Type: ICMP (ping requests)

Step 2: Generate Network Activity Data

The system processes incoming network packets and records activity.

- These packets act as input to the intrusion detection process

Step 3: Detection Command Execution

The Intrusion Detection Module monitors active network connections.

Command Used:

netstat -an

- The netstat command displays active network connections
- -a shows all connections
- -n shows numerical addresses and ports

Example Output (Network Connections)

```
tcp   0   0 127.0.0.1:22   127.0.0.1:54321   ESTABLISHED
tcp   0   0 127.0.0.1:22   127.0.0.1:54322   ESTABLISHED
tcp   0   0 127.0.0.1:22   127.0.0.1:54323   ESTABLISHED
```

Step 4: Output Analysis

The system analyzes connection data to identify patterns of suspicious activity.

Observation:

- Multiple connection attempts detected
- Same source and destination
- Occurring within a short time period

Step 5: Intrusion Detection

Repeated or unusual connection patterns indicate potential network-based intrusion activity.

Conclusion:

An intrusion event is detected due to abnormal network activity consistent with unauthorized access attempts.

Connection to Intrusion Detection Module

- Network traffic serves as system input
- Connection data provides visibility into activity
- Repeated connections indicate suspicious behavior
- Supports:
  - **FR-01: Intrusion Type Support (network-based intrusion)**
  - **FR-09: Attack Escalation**

5.4 Scenario #4 – Host-Based Intrusion Detection Using AIDE

This scenario demonstrates how the Intrusion Detection Module detects unauthorized changes to system files using AIDE (Advanced Intrusion Detection Environment).

Objective: To demonstrate how the Intrusion Detection Module detects host-based intrusions by comparing file states against a previously established baseline using AIDE.

Step 1: Input Creation (Baseline Initialization)

AIDE is used to create a baseline database of file attributes.

Steps:

1. Open terminal
2. Initialize the AIDE database:

```
aide --init
```

- The aide --init command creates a baseline snapshot of file attributes (e.g., size, permissions, checksum)

- This baseline represents the trusted state of the system

Example Input (Baseline State)

System Files: Monitored directories (e.g., /etc, /usr)

Attributes: size, permissions, checksum values

State: Trusted baseline

Step 2: Modify a File (Simulated Intrusion)

A file is modified to simulate an unauthorized change.

Steps:

echo "unauthorized change" >> sample.txt

- This command appends new content to the file
- The change represents a potential host-based intrusion

Example Input (Modified File)

File Name: sample.txt

Change: Content modified (additional text added)

Step 3: Detection Command Execution

The Intrusion Detection Module runs AIDE to detect changes.

Command Used:

aide --check

- The aide --check command compares the current system state to the baseline database
- Any differences are flagged as changes

Example Output (AIDE Detection)

Changed entries:

Sample.txt

Size: changed

SHA256: changed

Step 4: Output Analysis

The system processes AIDE output to identify unauthorized changes.

Observation:

- File modification detected
- Attributes (size, checksum) have changed
- File deviates from baseline

Step 5: Intrusion Detection

Because the file state differs from the baseline, the system determines that an intrusion has occurred.

Conclusion:

An intrusion event is detected due to unauthorized modification of a monitored file.

Step 6: Database Update (Post-Verification)

If the change is determined to be legitimate, the system updates the baseline.

Command Used:

```
aide --update
```

- Updates the baseline database to reflect trusted changes

Connection to Intrusion Detection Module

- AIDE provides file integrity monitoring
- Baseline comparison enables detection
- Output is processed by ThreatWatch
- Supports:
  - **FR-01: Intrusion Type Support (host-based intrusion)**
  - **FR-06: Response Capability** (update after decision)

6. Acceptance Criteria

Example 1

User story:

As a cybersecurity trainee, I want the system to evaluate the response I select for an intrusion alert so that I can learn whether my decision was correct.

Acceptance criteria:

- The system should evaluate the user's selected response after it is submitted.
- The system should determine whether the selected response is correct or incorrect based on the predefined scenario rules.
- The system should display the evaluation result to the user.
- The system should proceed to display the resulting state of the intrusion event after the response evaluation is completed.

Example 2

User story:

As a cybersecurity trainee, I want the system to escalate an intrusion event when I select an incorrect response so that I can understand the consequences of ineffective security decisions.

Acceptance criteria:

- If the user selects an incorrect response, the system should increase the severity level of the intrusion event.
- The system should simulate additional attack activity to represent escalation of the intrusion.

- The system should generate updated alerts reflecting the increased severity of the attack.
- The system should allow the user to continue responding to the escalated intrusion event.

7. Project Timeline

The development of ThreatWatch CyberLab follows a structured 15-week timeline to ensure all phases of the system are completed in a logical and organized manner. The project timeline includes planning, system design, development, integration, testing, and finalization phases.

A Gantt chart was created to visually represent task sequencing, dependencies, and deadlines across the semester. This chart ensures that all components of the system are developed in the correct order and completed within the designated timeframe.

7.1 Interactive Project Timeline

An interactive version of the project timeline, including task dependencies and updates, is available via Monday.com: https://gyniaps-team.monday.com/boards/18410248173

References

Indiana-University. (2020, February 11). *Indiana-University/Puppet-aide: This puppet module manages the installation and configuration of aide (advance intrusion detection environment)*. GitHub. https://github.com/indiana-university/puppet-aide

Monday.com. (2026). *Work management and Gantt chart tool.* Retrieved from https://monday.com